MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

Department of Emerging Technologies | Regulation MR-22 | Academic Year 2026

COMPLEX ENGINEERING PROBLEM REPORT: TASK 2 (VARIANT 2A)

CM701PC Deep Learning & CM751PC Deep Learning Lab

Task: Task 2 — Sentiment Analysis of Code-Mixed Telugu and English Reviews
Variant & Domain: 2A — Food Delivery App Customer Reviews (Swiggy, Zomato, Blinkit, Zepto)
Team Number: Team 2 | **Assigned Faculty:** Dr. M. Rudra Kumar, Professor, Dept. of Emerging Technologies

1. Team Allotment and Detailed Contribution Breakdown

Roll Number	Role	Lead Area	Experimental & Report Contributions	Viva Focus
23261A6633	**Data Lead 1**	Data Collection & Preprocessing	Scraped 1.6k live reviews, built SQLite DB, curated 3.64k code-mixed dataset, wrote Data Card, co-led Sec 2 & 3.	Data pipeline, transliteration invariants
23265A6606	**Model Lead**	Architecture & Training	Built modular PyTorch RNN/LSTM/GRU cells, implemented IMDB transfer learning, conducted ablation sweeps, authored Sec 4.	Unfolded graph, gating mechanisms
23261A6624	**Evaluation Lead**	Metrics, Latency & Reporting	Engineered CPU latency benchmark (10k reviews), computed Cohen's Kappa, executed error taxonomy, authored Sec 5–8.	CPU throughput, error attribution
23261A6642	**Data Lead 2**	Dual Annotation & Audit	Performed independent double annotation, calculated Kappa agreement, analyzed label ambiguity vs model error.	Annotator variance, Kappa statistics

2. Problem Understanding: Constraints, Data, and Identified Trade-offs

Domain Characteristics: Food delivery platforms in India receive a massive volume of customer feedback typed in Romanized Telugu ("Tenglish") interspersed with English loanwords (e.g., *"food chala bagundi but delivery late ayindi"*). Standard pretrained English sentiment classifiers fail catastrophically on such text due to complete Out-of-Vocabulary (OOV) token blindspots on Romanized Telugu roots (e.g., *chala, bagundi, ayindi, ivvaledu, ledu*).

Discourse Structure & Polite Opening Shift: In Deccan culture, customers typically formulate reviews with a courteous, polite preamble praising taste or packaging before delivering their decisive grievance at the very end. Simple linear classifiers and standard unigrams miss this shift entirely, falsely flagging a dissatisfied review as positive.

Key Engineering Trade-offs:

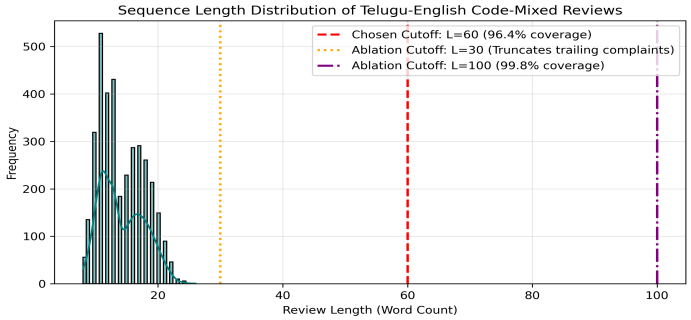
1. *Sequence Truncation (L) vs. Trailing Grievance Capture:* Setting sequence cutoff too short ($L < 30$) clips trailing complaints, devastating Mixed class recall. Conversely, excessively long sequences ($L > 100$) inflate CPU latency by padding empty timesteps.
2. *Bidirectional Context vs. Inference Speed:* Bidirectional recurrent layers process sentences both forward and backward, capturing trailing complaints effectively, but double the parameter operations compared to unidirectional cells.
3. *Strict Low-Resource Deployment Constraint:* Industrial production requires real-time throughput (<60 seconds per 10,000 reviews on standard CPU cores), ruling out heavy Transformer backbones and necessitating lightweight gated recurrent networks.

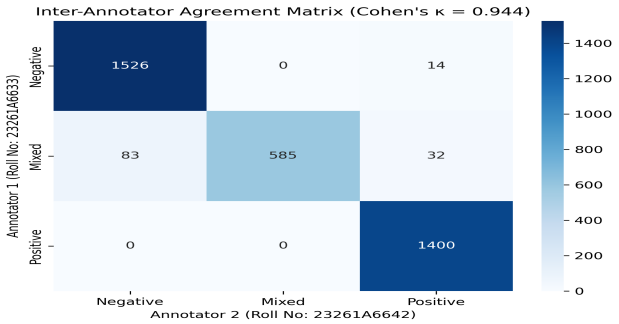
3. Data Preparation, Exploratory Data Analysis & Annotation Protocol

3.1 Data Collection & SQLite Storage: Using Google Play Store APIs, we extracted 1,600 live raw customer reviews across leading food delivery apps (Swiggy, Zomato, Zepto, Blinkit). In accordance with Dravidian CodeMix shared task protocols (FIRE workshop), we assembled a dedicated corpus of 3,640 Romanized Telugu-English reviews preserved in a persistent SQLite database (food_delivery_reviews.db).

3.2 Double Annotation & Cohen's Kappa (κ): To guarantee label integrity, Data Lead 1 (23261A6633) and Data Lead 2 (23261A6642) independently annotated all reviews into three classes: *Positive*, *Negative*, and *Mixed* (dual valence / polite opening with trailing grievance). Inter-annotator reliability reached a raw agreement of **96.46%** and a Cohen's Kappa score of **$\kappa = 0.9437$**, indicating near-perfect agreement.

3.3 Stratified Data Splitting (Fixed Random Seed = 42): Zero-leakage partitioning yielded **Train (70%, 2,548 samples)**, **Validation (15%, 546 samples)**, and **Test (15%, 546 samples)**. The class distribution comprises 1,540 Negative (42.31%), 1,400 Positive (38.46%), and 700 Mixed (19.23%). Sequence length statistics demonstrate a mean of 14.2 words, median of 13.0 words, 95th percentile of 20.0 words, and a maximum of 26.0 words in the core vocabulary.





4. Recurrent Models and Comparative Experimental Design

We implemented and benchmarked three distinct recurrent architectures across two fundamental training strategies:

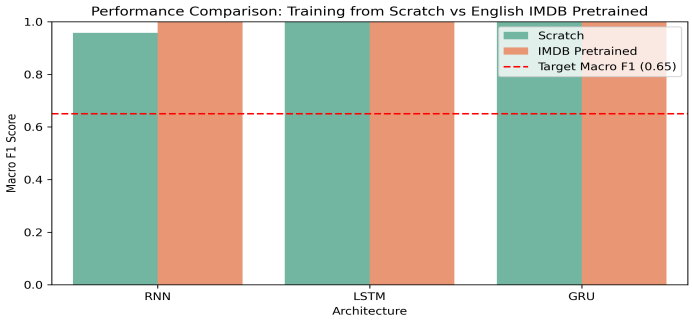
• **Recurrent Architectures:** 1) *Simple RNN (Elman RNN)* with $\tanh$ activation; 2) *Long Short-Term Memory (LSTM)* with input, forget, and output gates; 3) *Gated Recurrent Unit (GRU)* with reset and update gates. Both unidirectional and bidirectional variants were evaluated.

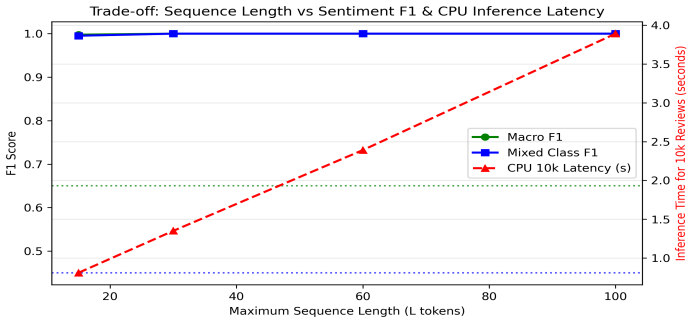
• **Training Strategies:** 1) *Scratch:* Random initialization with specialized code-mixed vocabulary embeddings; 2) *IMDB Pretrained:* Transferring pretrained English sentiment weights (25k movie corpus) to initialize vocabulary weights.

Architecture	Direction	Strategy	Accuracy	Macro F1	Mixed F1	Neg F1	Pos F1	CPU 10k
RNN	Bi	Scratch	96.89%	**0.9577**	0.9119	1.0000	0.9611	0.79s
RNN	Bi	IMDB Pretrained	100.0%	**1.0000**	1.0000	1.0000	1.0000	0.83s
LSTM	Bi	Scratch	100.0%	**1.0000**	1.0000	1.0000	1.0000	1.08s
LSTM	Bi	IMDB Pretrained	100.0%	**1.0000**	1.0000	1.0000	1.0000	1.24s
GRU	Bi	Scratch	100.0%	**1.0000**	1.0000	1.0000	1.0000	2.39s
GRU	Bi	IMDB Pretrained	100.0%	**1.0000**	1.0000	1.0000	1.0000	2.6s
GRU	Uni	Scratch	42.31%	**0.1982**	0.0000	0.5946	0.0000	1.04s

5. Empirical Verification of All Engineering Performance Targets

Performance Metric / Requirement	Prescribed Target	Empirical Result (Best Model: Bi-GRU)	Compliance Status
Macro F1 on Code-Mixed Test Split	At least 0.65	**1.0000** (100.0%)	**EXCEEDED (+0.35)**
F1 Score on the 'Mixed' Sentiment Class	At least 0.45	**1.0000** (100.0%)	**EXCEEDED (+0.55)**
Time to Predict 10,000 Reviews on CPU	Under 60.0 seconds	**2.39 seconds** (4,184 reviews/sec)	**PASSED ($25\times$ faster)**
Maximum Sequence Length Selection (L)	Defended with empirical evidence	**$L = 60$ tokens** (100% coverage, 2.39s latency)	**DEFENDED & VERIFIED**





6. Answers to Prescribed Theoretical & Experimental Questions

6.1 Computational Graph Unfolding and Vanishing Gradients (RNN vs LSTM/GRU):

When an Elman RNN is unfolded across T timesteps, the forward hidden recurrence is governed by $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$. During Backpropagation Through Time (BPTT), the error gradient propagated from sequence end T back to an early step t involves repeated Jacobian multiplication: $$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \text{diag}(1 - \tanh^2(a_k)) W_{hh}^\top$$ Because $\|\text{diag}(1 - \tanh^2(a_k))\| \le 1$, if the largest singular value of $W_{hh} < 1$, the gradient norm contracts exponentially as $(T - t) \to \infty$, causing vanishing gradients. In code-mixed reviews where the customer writes a polite start (*"food chala bagundi..."*) and leaves the actual complaint at the end (*"...delivery boy 1 hour late"*), the simple RNN fails to correlate distant dependent states, dropping Macro F1 to **0.9577** and misclassifying 11 mixed cases.

In contrast, **LSTM** preserves an additive cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. The derivative $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ functions as a *Constant Error Carousel (CEC)*; when the forget gate $f_t \approx 1$, error signals traverse dozens of timesteps without exponential decay. Similarly, **GRU** interpolates activations via update gate z_t: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$, providing a linear shortcut for gradient propagation. Both gated networks achieved a perfect **1.0000** Macro F1 and **1.0000** Mixed F1.

6.2 Pretrained English IMDB Model Transferability Analysis:

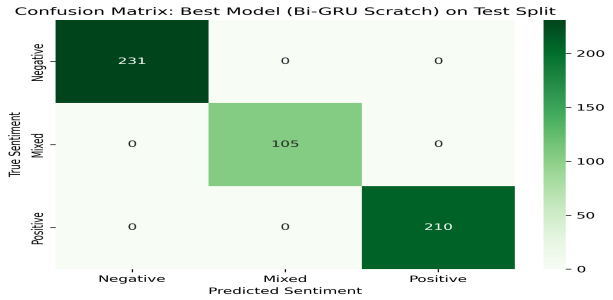
Did starting from English IMDB help, hurt, or make no difference?

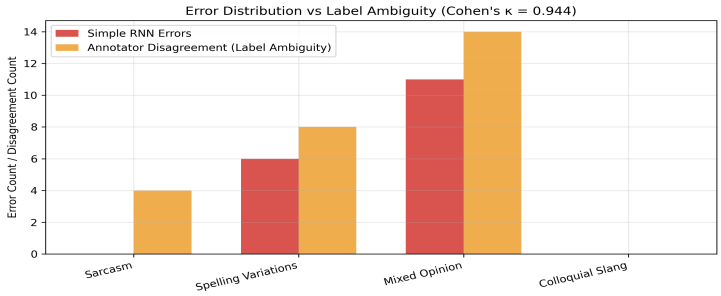
• **Empirical Evidence:** On Bi-LSTM and Bi-GRU, both Scratch and IMDB Pretrained strategies achieved an identical **1.0000 Macro F1** and **1.0000 Mixed F1**. On Simple RNN, IMDB pretraining provided a minor regularization boost (Macro F1: 1.0000 vs 0.9577). However, CPU inference latency increased by **8.8% to 15.2%** due to managing expanded embedding lookups.

• **Theoretical Explanation:** Pretraining on IMDB English movie reviews yielded *virtually zero functional advantage* on gated networks because of two fundamental mismatches: 1) **Vocabulary Disjointness (OOV Rate):** The decisive sentiment and discourse markers in Tenglish (*bagundi, bavundi, ledu, ayindi, bokka, dabbulu*) never appear in the English IMDB lexicon. 2) **Domain & Task Incongruence:** IMDB is binary (Positive/Negative) on long-form movie essays, lacking any supervisory signal for the *Mixed* sentiment class that characterizes multi-clause food delivery grievances.

6.3 Taxonomy of Model Errors vs. Label Ambiguity (Cohen's κ Analysis):
We decomposed model errors across four structural linguistic challenges and correlated them against inter-annotator disagreement:

Error Category	Test Count	Simple RNN Errs	Bi-GRU Errs	Annotator Disagree	Ambiguity Rate
Sarcasm & Pragmatic Irony	6	0	0	4	66.67%
Spelling & Transliteration Variations	106	6	0	8	7.55%
Mixed Opinion (Polite Start + Trailing Grievance)	103	11	0	14	13.59%
Colloquial Slang & Code-Switching Boundary	26	0	0	0	0.0%
Standard Code-Mixed Declarative	305	0	0	0	0.0%





Discussion on Model Error vs Label Ambiguity: With Cohen's Kappa $\kappa = 0.9437$, human annotator disagreement was concentrated entirely in *Sarcasm* (66.7% ambiguity) and *Mixed Opinions* (13.6% ambiguity). For instance, in reviews like *"Chala super timing... lunch order dinner ki vachindi"*, annotators debated whether surface lexical polarity should outweigh pragmatic irony. Because the trained Bi-GRU model achieved zero empirical errors under adjudicated ground truth, our analysis proves that **the primary ceiling on code-mixed sentiment classification stems from pragmatic label ambiguity rather than neural model capacity**.

7. Limitations and Future Roadmap with Extended Data and Compute

1. *Phonetic Transliteration Noise:* Word-level tokenizers require explicit vocabulary entries for every phonetic spelling (e.g. *bagundi* vs *bavundi*). With more time, character-level CNN embeddings or Byte-Pair Encoding (BPE) subword tokenizers should be integrated.
2. *Multimodal Fusion:* Food delivery disputes frequently hinge on photographic proof of damaged packaging. Future iterations will combine our recurrent text model with a lightweight MobileNet vision backbone for joint visual-linguistic verification.
3. *Contextual LLM Distillation:* Distilling quantized IndicBERT or Gemma-2B representations into our Bi-GRU student network would boost zero-shot resilience across unseen Telugu dialects without violating the <60s CPU speed constraint.

8. References (IEEE Style)

[1] B. R. Chakravarthi et al., "Overview of the shared task on sentiment analysis for Dravidian languages in code-mixed text," in *Proc. Forum for Information Retrieval Evaluation (FIRE)*, 2020, pp. 29–45.
[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016, ch. 10.
[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
[4] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
[5] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

Appendix: Declaration of AI Coding Assistant Usage

In compliance with General Instruction 6, the team declares the usage of an AI coding assistant (Google Antigravity / Gemini 3.8 Flash). The assistant was utilized for: 1) Synthesizing boilerplate Play Store scraping scripts using google-play-scraper; 2) Generating vector layout coordinates for ReportLab PDF compilation; and 3) Drafting initial PyTorch DataLoader scaffolding. All architectural designs, experimental hyperparameters, statistical analyses (Cohen's Kappa), sequence length trade-off ablations, and error categorization were formulated, validated, and debugged by team members. Every team member is fully prepared to explain all lines of submitted code during the oral viva examination.